

moves to solve. This means a 31-disk tower requires over a billion moves to complete!

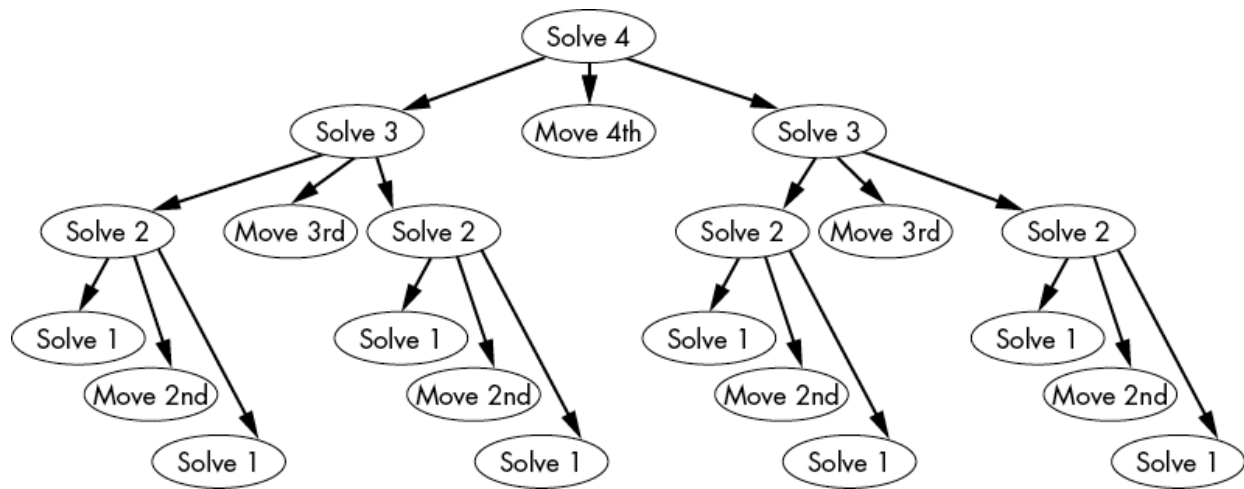


Figure 3-5: The series of operations for solving a four-disk Tower of Hanoi

Let's ask ourselves the three questions for creating a recursive solution:

What is the base case? Solving a tower of one disk.

What argument is passed to the recursive function call? Solving a tower of size one less than the current size.

How does this argument become closer to the base case? The size of the tower to solve decreases by one disk for each recursive call until it is a one-disk tower.

The following *towerOfHanoiSolver.py* program solves the Tower of Hanoi puzzle and displays a visualization of each step:

```

import sys

# Set up towers A, B, and C. The end of the list is the top o
f the tower.
TOTAL_DISKS = 6 ❶

# Populate Tower A:
TOWERS = {'A': list(reversed(range(1, TOTAL_DISKS + 1))), ❷
          'B': [],
          'C': []}

def printDisk(diskNum):

```